

Theory of Computing :

12. Decidability



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- Algorithms and Church's Thesis
- Decidable Languages & Examples
- Diagonalization Method & Infinities
- Proving by Reducibility
- Undecidability of the Halting Problem
- Post Correspondence Problem



**There is a solution
to every problem ?**



Universal Turing Machine

- You write a programming code in new Language **X** , it does some computation and return some input.

Universal Turing Machine



- You write a programming code in new Language **X** , it does some computation and return some input.
- You need to compile and execute the program written in X by compiler written in which language ?

Universal Turing Machine

- Given any Turing Machine **M** to conduct some computation, there is the Universal Turing Machine to simulate or execute M for any given input w
- The universal Turing Machine **U** :
 - *Halt iff M halts on input w .*
 - *If M is a deciding/semi-deciding machine, then*
 - *If M accepts, accept.*
 - *If M rejects, reject.*
 - *If M computes a function, then $U(\langle M, w \rangle)$ must equal $M(w)$*

Universal Turing Machine

- We can construct a universal TM that accepts the language $L = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \in L(M) \}$
 - Given an **encoded representation** of the turing machine M with an input string w
 - The word w is accepted by the language represented by the turing machine M

Universal Turing Machine

- We can construct a universal TM that accepts the language

$L = \{ \langle M, w \rangle \mid$

- M is a Turing Machine with
- w is a word accepted by M
- The word w is accepted by the language represented by the Turing machine M

The notation $\langle M, w \rangle$: The encoded representation of :

Turing Machine accepting the word w

Algorithms and Church's Thesis



- Turing Machines are an abstract model of computation, their purpose is to define in a mathematical way what problems are theoretically computable and which are not.
- In 1900, mathematician David Hilbert identified **23 mathematical problems** and posed them as a challenge for the coming century.
- Hilbert's tenth problem was to devise an **algorithm** that tests whether a **polynomial** has an integral root. (*Integers to be assigned to the polynomial variables to reach a value of zero*)

Algorithms and Church's Thesis



- Hilbert did not use the term algorithm but rather “a process according to which it can be determined by a finite number of operations.”
 - He assumed that such an algorithm must exist—someone need only find it.
 - But : it is algorithmically unsolvable.
- Church–Turing thesis provides the definition of algorithm necessary to resolve Hilbert's tenth problem

Algorithms and Church's Thesis



- Church-Turing thesis. There is an “effective procedure” for a problem if and only if there is a TM for the problem.
 - For a given problem, if we can write the algorithm or construct the turing machine, it is computable or algorithmically solvable

Algorithms and Church's Thesis



- Church-Turing thesis. There is an “effective procedure” for a problem if and only if there is a TM for the problem.
 - For a given problem, if we can write the algorithm or construct the turing machine, it is computable or algorithmically solvable
 - We are speaking always about the general cases, as there are unsolvable/uncomputable problems, but there are **instances** of that problem that we **can solve**.

Algorithms and Church's Thesis



- What's an Algorithm :
 - An algorithm is an effective/systematic/mechanical method for achieving the desired result for a given problem.
 - The desired result can be :
 - Decisional : Yes or No Answer
 - Computational : Conduct some computational and outputs the result.

Algorithms and Church's Thesis



- What's an Algorithm :
 - An algorithm is an effective/systematic/mechanical method for achieving the desired result for a given problem.
 - It has a **finite** number of instructions.
 - If carried out without error, it produces the desired result in a finite number of steps.
 - It can be carried out by a human with only paper and pen ?

Algorithms and Church's Thesis

- What's an Algorithm :
 - An algorithm is an effective/systematic/mechanical method for achieving a given goal.
 - It should work for **all** input instances from a given domain
 - It can be carried out by a human with only paper and pen ?
 - It should work for all input instances from a given domain

Algorithms and Church's Thesis

- What's an Algorithm :

- An algorithm is an effective / constructive / mechanical method for

ach

Hypothesis : Any algorithm can be represented and executed on a Turing Machine

■
■ result in a

finite number of steps.

- It can be carried out by a human with only paper and pen ?

Decidable Languages & Examples



- The collection of strings that M **accepts** is the language of M , or the language recognized by M
 - A language is called **Turing-recognizable** if some Turing machine recognizes it
 - Mainly : Accepting words that belong to the language.
 - For words not in the language:
 - Reject **or** Loop **forever..**

Decidable Languages & Examples



- **Turing-decidable** language or simply decidable if some Turing machine decides it
 - Halts and Accepts for words in the language
 - Halts and Reject for words not in the language
- Every Decidable language is also recognizable.

Decidable Languages & Examples



- **Problem** : Given a DFA called B, and given another word w . is the Acceptance problem decidable ?
 - Algorithmically Computable
 - Halts with either : **Accept or Reject.**
- *Remember : General case = ALL instances of problems*

Decidable Languages & Examples

- Problem

Acceptance

○

○

A language is decidable if there exists a Turing machine (or an algorithm) that halts on all inputs and correctly decides whether the input string belongs to the language.

the

- Remember : General case = ALL instances of problems

Decidable Languages & Examples

- **Problem** : Given a DFA called B, and given another word w , is the Acceptance problem decidable ?
 - Computable
 - Halts with either : Accept or Reject.
- The problem is expressed as a Language:

$$A_{\text{DFA}} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w \}$$

Decidable Languages & Examples

- **Problem** : Given a DFA called B, and given another word w, is the

Accepted?

- Complexity
- Hardness

Showing that the language is decidable is the same as showing that the computational problem is decidable

- The problem is expressed as a Language

$$A_{\text{DFA}} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w \}$$

Decidable Languages & Examples

- **Problem** : Given a DFA called B , and given another word w , is the Acceptance problem decidable ?
- We simply need to present a Turing Machine M that decides A_{DFA} .
 - $M =$
 - *On input $\langle B, w \rangle$, where B is a DFA and w is a string:*
 - *Simulate B on input w .*
 - *If the simulation ends in an accept state, “**accept**”. If it ends in a non-accepting state, “**reject**”*

PROOF We mention just a few implementation details of this proof. For those of you familiar with writing programs in any standard programming language, imagine how you would write a program to carry out the simulation.

First, let's examine the input $\langle B, w \rangle$. It is a representation of a DFA B together with a string w . One reasonable representation of B is simply a list of its five components: Q , Σ , δ , q_0 , and F . When M receives its input, M first determines whether it properly represents a DFA B and a string w . If not, M rejects.

Then M carries out the simulation directly. It keeps track of B 's current state and B 's current position in the input w by writing this information down on its tape. Initially, B 's current state is q_0 and B 's current input position is the leftmost symbol of w . The states and position are updated according to the specified transition function δ . When M finishes processing the last symbol of w , M accepts the input if B is in an accepting state; M rejects the input if B is in a nonaccepting state.

non-accepting state, reject

Decidable Languages & Examples

- **Problem** : Given a DFA called B , and given another word w , is the Acceptance problem decidable ?
- Steps for the Turing Machine M
 1. *Verification of Input Syntax for the encoded string $\langle B, w \rangle$ such that B can be represented as a list containing 5 elements Q, Σ, δ, q_0 , and F*
 2. *Store the initial state on the memory tape at some position that you can return easily*
 3. *Start reading the symbols of the word w and for each symbol you read, find the relevant transitions.*
 4. *Update the current state*
 5. *When you complete reading all symbols of the word w , examine if the current state is accepting $\rightarrow$ Accept, otherwise, Reject.*

Decidable Languages & Examples

- **Problem** : Given a DFA called B , and given another word w , is the Acceptance problem decidable ?
- Steps for the Turing Machine M
 1. *Verification of Input Syntax for the encoded string $\langle B, w \rangle$ such that B can be represented as a list containing 5 elements Q, Σ, δ, q_0 , and F*
 2. *Store the initial state on the memory tape at some position that you can return easily*
 3. *Start reading the symbols of the word w and for each symbol you read, **find the relevant transitions.***
 4. *Update the current state*
 5. *When you complete reading all symbols of the word w , examine if the current state is accepting $\rightarrow$ Accept, otherwise, Reject.*

Decidable Languages & Examples

- **Problem** : Given a DFA called B , and given another word w , is the Acceptance problem decidable ?

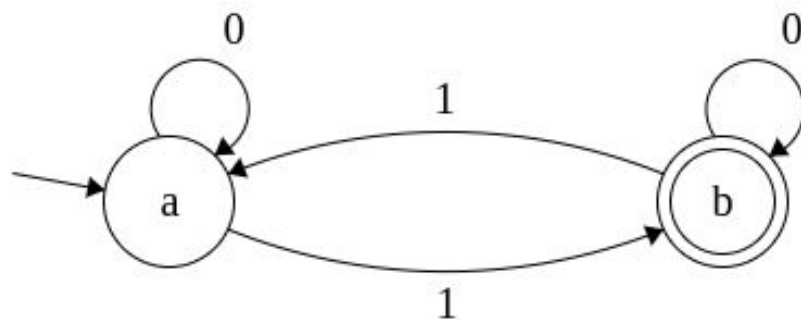
- Steps for

1. Verify the word w is represented correctly
2. Store the current state in a variable
3. Start reading the symbols of the word w and for each symbol you read, find the relevant transitions.
4. Update the current state
5. When you complete reading all symbols of the word w , examine if the current state is accepting → Accept, otherwise, Reject.

The better encoding you propose for the problem, the easier it will be for constructing the TM

Decidable Languages & Examples

- **Problem** : Given a DFA called B, and given another word w , is the Acceptance problem decidable ?
 - A_{DFA} is a decidable language



Decidable Languages & Examples

- **Problem** : Given a DFA called B , and given another word w , is the Acceptance problem decidable ?
 - A_{DFA} is a decidable language

a	^	a	0	a	,	a	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	1	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Examples

- Problem

Acceptance

- A_{DFA}

Let's see if the TM can simulate 10 by the DFA ?

a	^	a	0	a	,	a	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	1	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	a	0	a	,	a	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	1	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	a	0	a	,	a	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	a	0	a	,	a	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	A	0	a	,	A	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	A	0	a	,	A	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	A	0	a	,	A	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	A	0	a	,	A	1	b	,	b	1	a	,	b	0	b	#	b	/	a	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language



a	^	A	0	a	,	A	1	b	,	b	1	a	,	b	0	b	#	b	/	b	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Example

Let's see if the TM can simulate 10 by the DFA ?

- Problem

Acceptance problem decidable ?

- A_{DFA} is a decidable language

a	^	a	0	a	,	a	1	b	,	b	1	a	,	b	0	b	#	b	/	b	~	N	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Decidable Languages & Examples

- **Problem** : Given a DFA called B, and given another word w , is the Acceptance problem decidable ?
 - **A_{DFA} is a decidable language.**
 - DFA will always terminates upon finishing reading a finite number of characters in linear fashion.

Decidable Languages & Examples



- **Problem** : Given a NFA called B , and given another word w , is the Acceptance problem decidable ?

Decidable Languages & Examples



- **Problem** : Given a NFA called B , and given another word w , is the Acceptance problem decidable ?
 - We convert the NFA to DFA
 - We follow the same procedure described for the DFA to arrive that the acceptance problem for the NFA is computable

Decidable Languages & Examples

- **Problem** : Given a NFA called B, and given another word w , is the Acceptance problem decidable ?
 - We convert the NFA to DFA
 - We follow the same procedure described for the DFA to arrive that the acceptance problem for the NFA is computable
- **Same Problem** : Can we generate such a string w using a given regular expression : **Generation Problem** ?

Decidable Languages & Examples

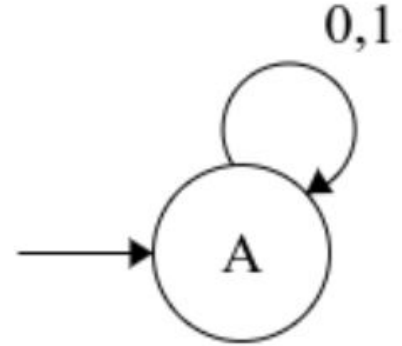


- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ? "***Emptiness testing***"

Decidable Languages & Examples

- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ?

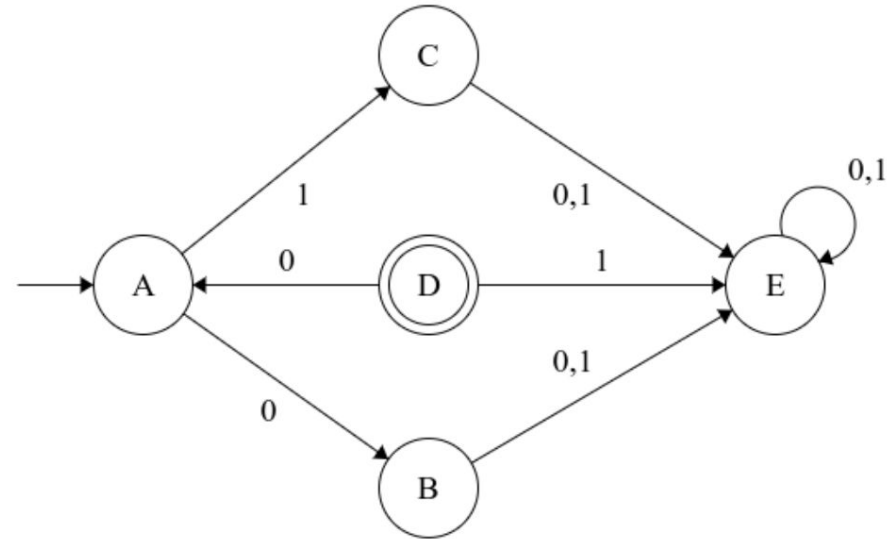
An SINGLE Instance of the problem is the shown DFA along with the word 101



Decidable Languages & Examples

- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ?

Does this instance of the DFA represents the language of the empty set



Decidable Languages & Examples

- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ?
- The problem is represented as a Language E_{DFA}
 $E_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}.$

Decidable Languages & Examples

- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ?
- The problem is represented as a Language E_{DFA}
 $E_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}.$
- *A DFA accepts some string iff reaching an accept state from the start state by traveling along the arrows of the DFA is possible*

Decidable Languages & Examples

- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ?
- We simply need to present a Turing Machine T that decides E_{DFA} .
 - T = On input $\langle A \rangle$, where A is a DFA
 1. *Mark the start state of A .*
 2. *Repeat until no new states get marked:*
 - *Mark any state that has a transition coming into it from any state that is already marked*
 3. *If no accept state is marked, accept ; otherwise, reject ."*

Decidable Languages

Examples

Same Algorithm as the
Problem of Connected
Graphs

- **Problem** : Given a DFA for the empty set, is it computable for any given string ? it is decidable ?
- We simply need to present a Turing Machine T that decides E_{DFA} .
 - T = On input $\langle A \rangle$, where A is a DFA
 1. *Mark the start state of A .*
 2. *Repeat until no new states get marked:*
 - *Mark any state that has a transition coming into it from any state that is already marked*
 3. *If no accept state is marked, accept ; otherwise, reject ."*

Decidable Languages & Examples



- **Problem** : Given two DFAs, can we algorithmically compute or decide if they are equivalent ?

Decidable Languages & Examples

- **Problem** : Given two DFAs, can we algorithmically compute or decide if they are equivalent ?
- The problem whether two DFAs recognize the same language is decidable, is represented as a language EQ_{DFA} such that

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B) \}.$$

Decidable Languages & Examples



- **Problem** : Given two DFAs, can we algorithmically compute or decide if they are equivalent ?
- We construct a new DFA C from A and B, where C accepts only those strings that are accepted by **either** A **or** B but **not by both**
 - **If A equals B , it implies that C is nothing.**

Decidable Languages & Examples

- **Problem** : Given two DFAs, can we algorithmically compute or decide if they are equivalent ?
- We construct a new DFA C from A and B, where C accepts only those strings that are accepted by **either** A **or** B but **not by both**
 - **If A equals B , it implies that C represents the empty set.**
- Language for C can be represented as:

$$L(C) = \left(L(A) \cap \overline{L(B)} \right) \cup \left(\overline{L(A)} \cap L(B) \right)$$

Decidable Languages & Examples

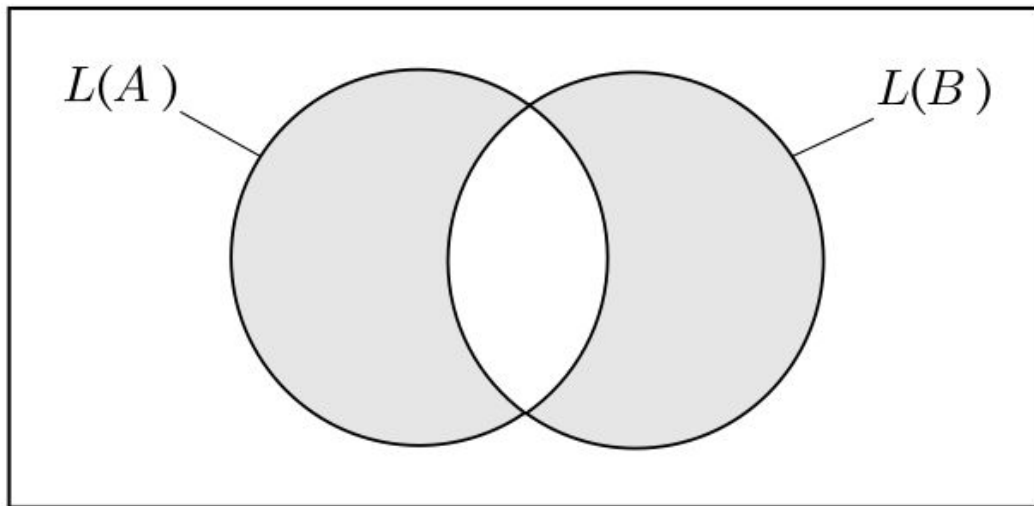
- **Problem :**

if they are

- We construct

strings that

- **If A is**



pute or decide

pts only those

both

pty set.

- Language for C can be represented as:

$$L(C) = \left(L(A) \cap \overline{L(B)} \right) \cup \left(\overline{L(A)} \cap L(B) \right)$$

Decidable Languages & Examples

- **Problem** : Given two DFAs, can we algorithmically compute or decide if they are equivalent ?
- Language for C can be represented as:

$$L(C) = \left(L(A) \cap \overline{L(B)} \right) \cup \left(\overline{L(A)} \cap L(B) \right)$$

- The construction of C using Union, Complement , Intersection are algorithms that can be carried out by Turing machines
- *Provided that we prove that such constructions are closed under the class of regular languages...*

Decidable Languages & Examples

- **Problem** : Given two DFAs, can we algorithmically compute or decide if they are equivalent?

- Language for

$$L(C) \cap L(B)$$

To prove that $L(C)$ is the empty set, use the case discussed earlier

- The construction of C using Union, Complement, Intersection are algorithms that can be carried out by Turing machines

Decidable Languages & Examples



- Other problems for CFG:
 - Is generation of words using Context-Free Grammar decidable ?
 - Is the equivalence of two CFG computable ?
 - Is the emptiness testing for a language represented by CFG computable ?

Diagonalization Method & Infinities



- For Turing Machines : is the acceptance problem decidable for Turing Machine ?
 - $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \} .$

Diagonalization Method & Infinities



- For Turing Machines : is the acceptance problem decidable for Turing Machine ?
 - $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \} .$
- A_{TM} is **undecidable**

Diagonalization Method & Infinities

- For
Ma
o
• AT
- For During

**Another different Question:
Can we represent any problem or any
language using Turing Machine ?**

Diagonalization Method & Infinities



Time to forget Turing Machines

Diagonalization Method & Infinities



- Comparing two sets:
 - If two sets of finite size,
 - For instance : $\{a,b,c,d\}$ and $\{1, 2, 3, 4\}$:
 - Question :
 - How to determine that they have the same size
- (Assume we don't know how to count)**

Diagonalization Method & Infinities



- Comparing two sets:
 - If two sets of finite size,
 - For instance : $A=\{a,b,c,d\}$ and $B=\{1, 2, 3, 4\}$:
 - Answer :
 - For each element x in A , find a mapping or correspondence y in B such that y is mapped only by x

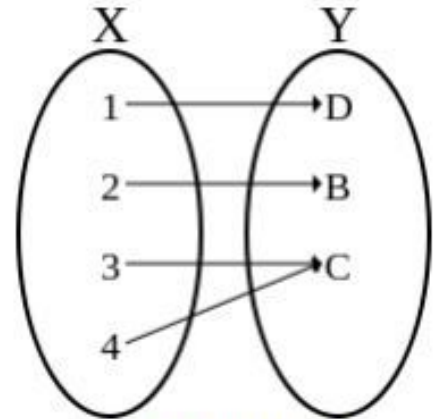
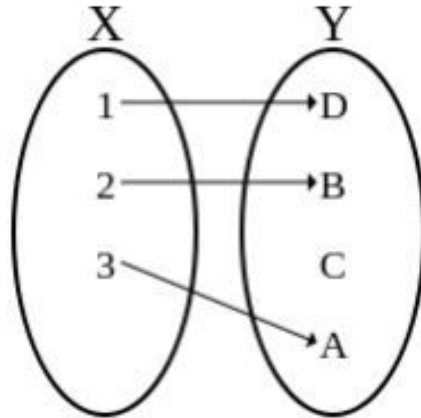
Diagonalization Method & Infinities



- Definitions:
 - **One-to-One function** : A function f which maps two sets: A to B.
 - If it never maps two different elements to the same place.
 - That is, if $f(a) \neq f(b)$ whenever $a \neq b$.
 - Other terminology : **injective**

Diagonalization Method & Infinities

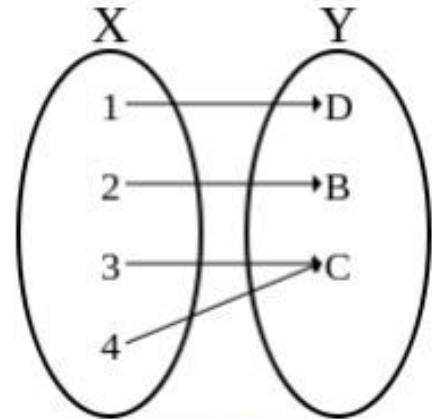
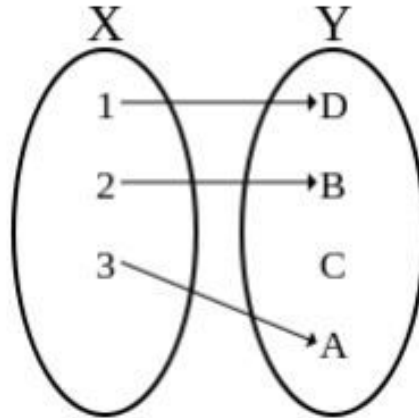
- Definitions:
 - **One-to-One function** : A function f which maps two sets: A to B.
 - If it never maps two different elements to the same place
 - That is, if f
 - Other termino



Diagonalization Method & Infinities

- Definitions:

- **One-to-One function** f which maps X to Y .
 - If it never maps two different elements to the same element.
 - That is, if $f(x) = f(y)$ then $x = y$.
- Other terminology: **Injective**.



Diagonalization Method & Infinities



- Definitions:
 - **Onto Function** : A function f which maps two sets: A to B
 - If it hits every element of B
 - That is, if for every $b \in B$, there is an $a \in A$ such that $f(a) = b$.
 - Other terminology : **surjective**

Diagonalization Method & Infinities

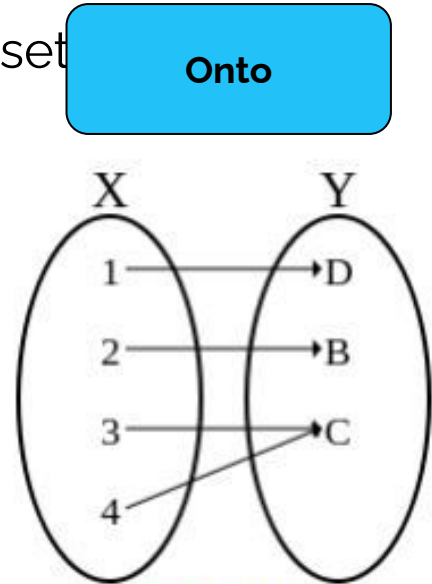
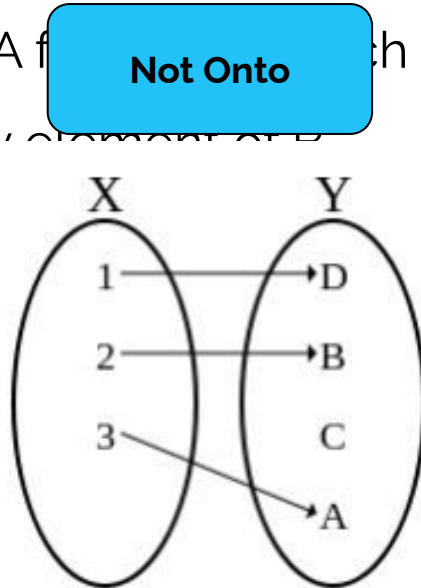
- Definitions:

- **Onto Function** : A function f which maps two sets

- If it hits every element of B

- That is, if f

- Other termino



Diagonalization Method & Infinities



- Definitions:
 - **Correspondence Function** is both **one-to-one** and **onto**.
 - Two sets have the same size if there is a correspondence function $f : A \rightarrow B$.
 - Every element of A maps to a unique element of B and each element of B has a unique element of A mapping to it.
 - A correspondence is simply a way of pairing the elements of A with the elements of B.
 - Other terminology : **bijjective**

Diagonalization Method & Infinities

- Definition

-

-

What if A has more elements than B ?

Impossible as A would be fed into the mapping function unless there are cases like division by zero

-

- Other terminology : **bijective**

Diagonalization Method & Infinities



- Given now an infinite set :
 - Example : Natural numbers $\{0,1,2,3,\dots\}$
- Question:
 - Can we count the elements in such set ?

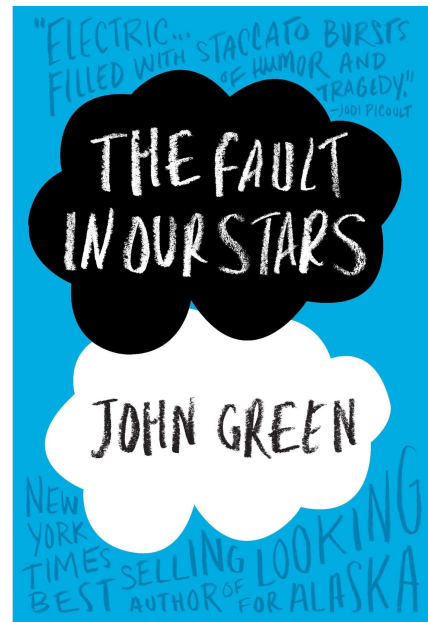
Diagonalization Method & Infinities

- Given two infinite sets :
 - Example :
 - Natural numbers $\{1,2,3,\dots\}$
 - Odd numbers $\{1,3,5,\dots\}$
- Question:
 - Can an infinite set be bigger than another infinite set ?

Diagonalization Method & Infinities

Reading the **Novel** written by John Green :

"There are infinite numbers between 0 and 1. There's .1 and .12 and .112 and an infinite collection of others. Of course, there is a bigger infinite set of numbers between 0 and 2, or between 0 and a million. Some infinities are bigger than other infinities... I cannot tell you how grateful I am for our little infinity. You gave me forever within the numbered days, and I'm grateful. "



Diagonalization Method & Infinities

- **Cantor Definition :**

- A set A is countable if either it is finite or it has the same size as $\mathbb{N}$ ($\mathbb{N}$ is the set of natural numbers)

Diagonalization Method & Infinities



- **Cantor Definition :**

- A set A is countable if either it is finite or it has the same size as $\mathbb{N}$ ($\mathbb{N}$ is the set of natural numbers)
 - There is a **correspondence function** between the set of natural numbers and the set A .
- *Georg Cantor (1848-1918) is a german mathematician. He was interested in the problem of measuring the sizes of infinite sets*

Diagonalization Method & Infinities

- Cantor Definition :

- A set A is countable if it has the same cardinality as $\mathbb{N}$ (

If we can enumerate the elements in the set , it is countable

- *Georg Cantor (1848-1918) is a german mathematician. He was interested in the problem of measuring the sizes of infinite sets*

Diagonalization Method & Infinities



- **Question** : is the set of even numbers **countable** ? or does it have the same size as $\mathbb{N}$

Diagonalization Method & Infinities

- **Question** : is the set of even numbers **countable** ? or does it have the same size as $\mathbb{N}$
- **Answer** :
 - Let :
 - $\mathbb{N}$ be the set of natural numbers $\{1, 2, 3, \dots\}$
 - E be the set of even natural numbers $\{2, 4, 6, \dots\}$.
 - Using Cantor's definition of size, there is a correspondence function **$f(n)=2n$**

Diagonalization Method & Infinities

- **Question** : is the set of even numbers **countable** ? or does it have the same size as $\mathbb{N}$?

- This is a simple visualization for the correspondence function

al numbers $\{1, 2$

■ E be the set of even natural number

- Using Cantor's definition of size, there is

function **$f(n)=2n$**

n	$f(n)$
1	2
2	4
3	6
$\vdots$	$\vdots$

Diagonalization Method & Infinities

- **Question** : is the set of even numbers **countable** ? or does it have the same size as $\mathbb{N}$

- It may seem strange that even E is proper subset of $\mathbb{N}$, they have the same size, but as long we can pair any member from $\mathbb{N}$ to E , they have the same size

- Using Cantor's definition of size, there is

function $f(n)=2n$

(n)
2
4
6
$\vdots$

Diagonalization Method & Infinities

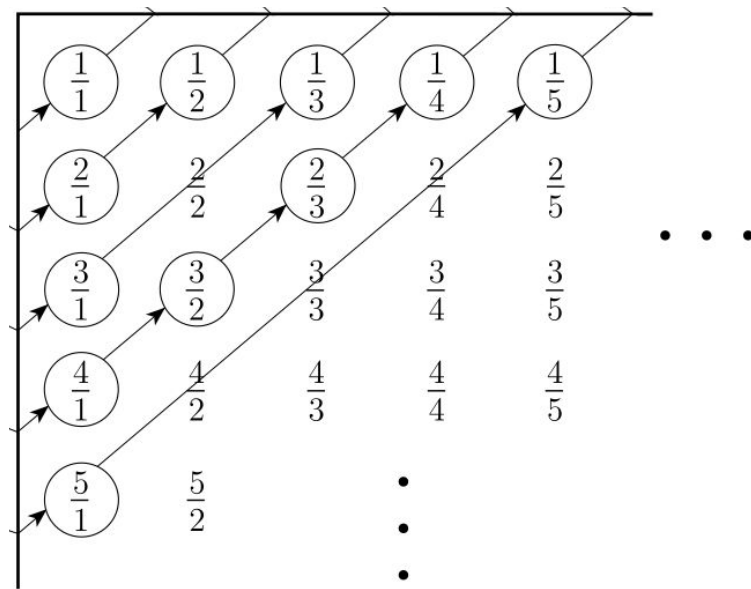
- **Question** : is the set of rational numbers **countable** ? or does it have the same size as $\mathbb{N}$

$$\mathbb{Q} = \left\{ \frac{m}{n} \mid m, n \in \mathbb{N} \right\}$$

- Can we find a correspondence function between them ?

Diagonalization Method & Infinities

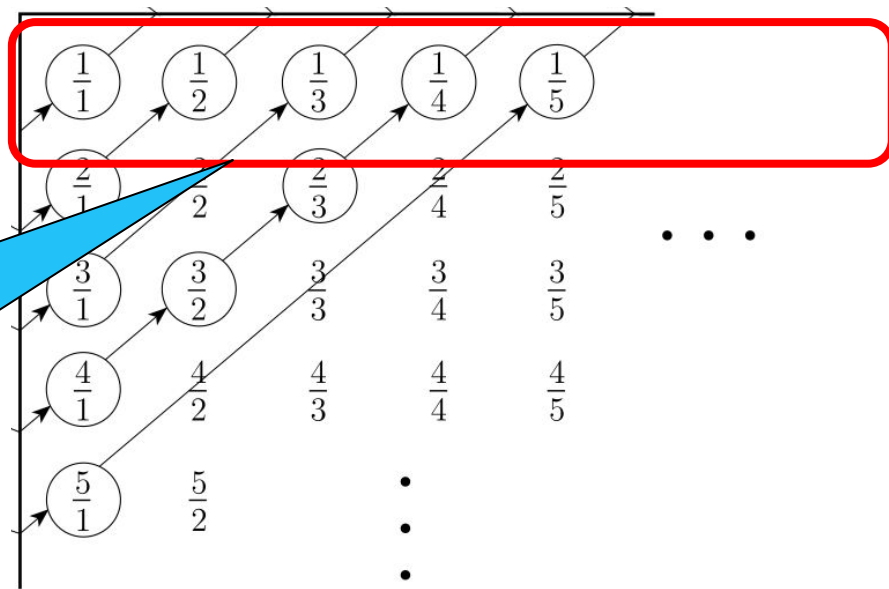
- **Question** : is the set of rational numbers **countable** ? or does it have the same size as $\mathbb{N}$



Diagonalization Method & Infinities

- **Question** : is the set of rational numbers **countable** ? or does it have the same size as $\mathbb{N}$

Each row corresponds to a given number placed as a numerator



Diagonalization Method & Infinities

- **Question** : is the set of rational numbers **countable** ? or does it have the same size as $\mathbb{N}$

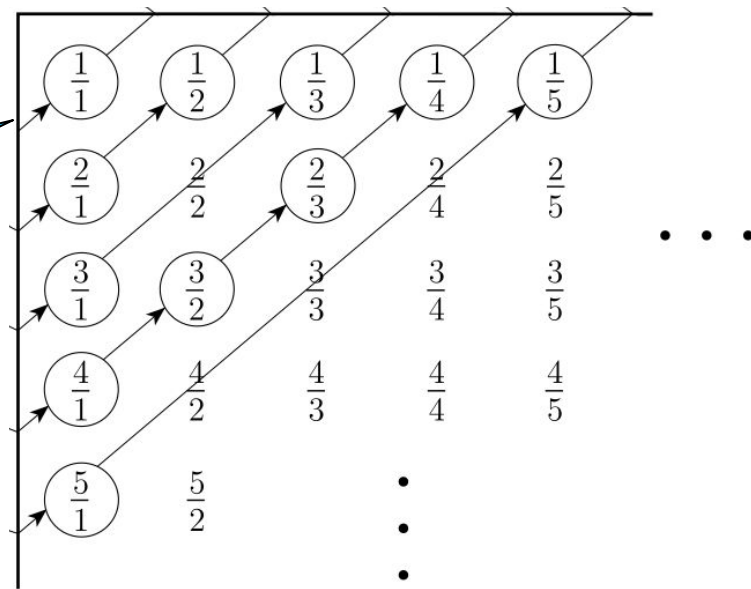
Each column corresponds to a given number placed as a denominator



Diagonalization Method & Infinities

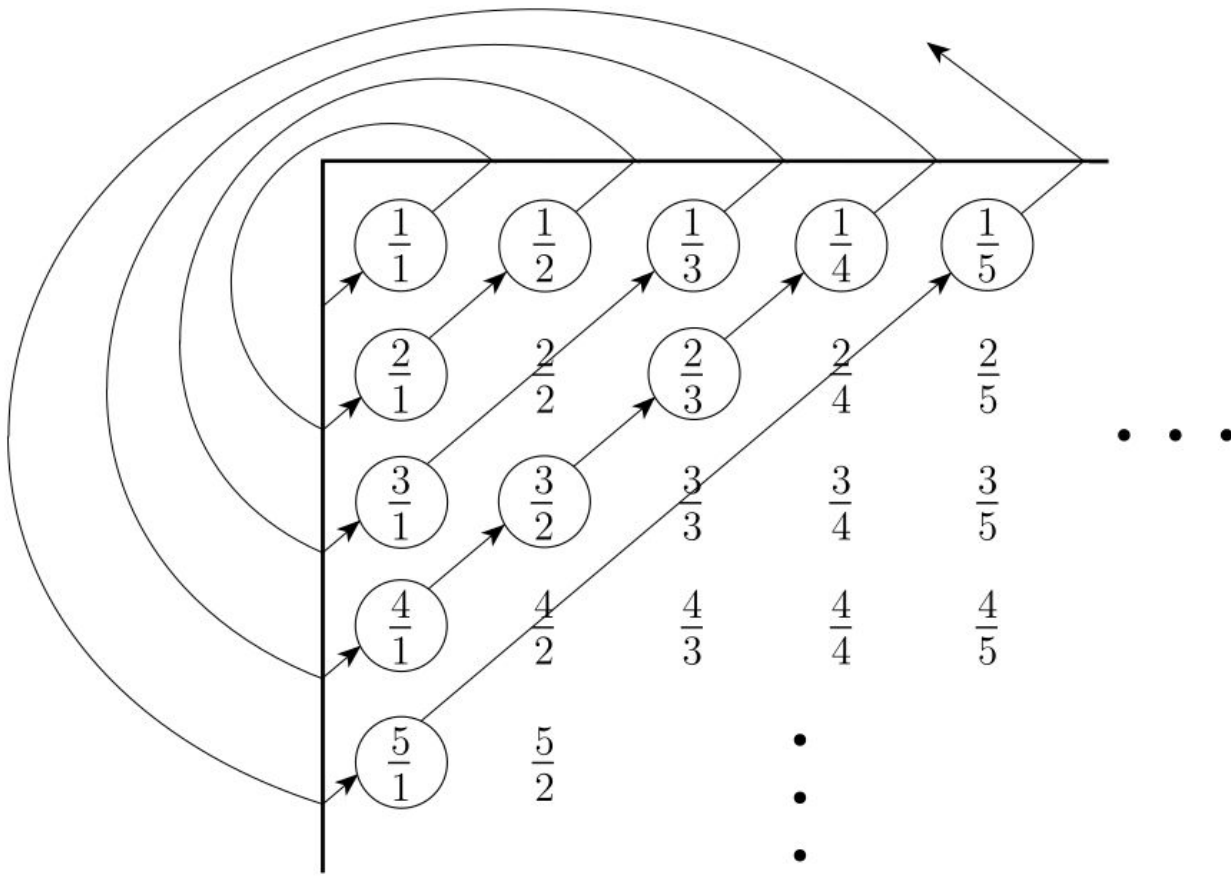
- **Question** : is the set of rational numbers **countable** ? or does it have the same size as $\mathbb{N}$

This way, ALL rational numbers are represented.
Can we enumerate them ?



© 2005 Blackwell Publishing Ltd *Journal of Internal Medicine* 258: 103–110

n	f(n)
1	$1/1=1$
2	$2/1=2$
3	$1/2$
4	$3/1=3$
5	$1/3$
..	..



Diagonalization Method & Infinities



- **Question** : is the set $\mathbb{R}$ of real numbers **countable** ? or does it have the same size as $\mathbb{N}$
 - Find a correspondence function
 - Same as Rational Number ?

Diagonalization Method & Infinities

- **Diagonalization :**
 - Given the following list of words,
 - Give a word not from the list.

?	?	?	?	?
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	O	X

Diagonalization Method & Infinities

- **Diagonalization :**
 - Given the following list of words,
 - Give a word not from the list.
 - Hello ?

H	E	L	L	O
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	O	X

Diagonalization Method & Infinities

- **Diagonalization :**
 - Given the following list of words,
 - Give a word not from the list.
 - What's the **Algorithm ?**

?	?	?	?	?
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	O	X

Diagonalization Method & Infinities

- **Diagonalization :**
 - Given the following list of words,
 - Give a word not from the list.
 - What's the **Algorithm ?**

But **guarantees** giving words
not form that table

?	?	?	?	?
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	O	X

Diagonalization Method & Infinities

- **Diagonalization :**

- Algorithm

- Sequentially:

- For each letter from the diagonal, choose a different letter, Possibly the next next letter

- Example :

at step 1: we choose R

R	?	?	?	?
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Diagonalization :**

- Algorithm

Will our new word (R????) be the same as the first word : QUIET ?
Never,

Because they have different first letter

at step 1: we choose R

onal,
sibly

R	?	?	?	?
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Diagonalization :**

- Algorithm

- Sequentially:

- For each letter from the diagonal, choose a different letter, Possibly the next next letter

- Example :

at step 2: we choose U

R	U	?	?	?
---	---	---	---	---

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Diagonalization :**

- Algorithm

Will our new word (**RU**???) be the same as the second word : **STONE** ?
Never,

Because they have different second letter

at step 2: we choose U

onal,
sibly

RU

1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- Diagonalization :

- Algorithm

At step n , Will our new word be the same as the n^{th} word
Never,

Because they have different n^{th} letter

at step 2: we choose U

onal,
sibly

	R	U	?	?	?
1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Diagonalization :**
 - This table for only 5 words ? does this method of "diagonalization" work for an infinite set ?

	R	U	G	B	Y
1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Diagonalization :**

- This table for only 5 words ? does this method of “diagonalization” work for an infinite set ?

- Each **ROW** corresponds to an existing word, we ensure to change the symbol for the constructed new word
⇒ therefore, it would never be in the list

	R	U	G	B	Y
1	Q	U	I	E	T
2	S	T	O	N	E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Diagonalization :**

- This table for only 5 words ? does this

The new word cannot be on the list: it is different from first word in first letter, different from second word in second letter, etc.

- **It works !** Each column corresponds to an existing word, we ensure to change the symbol for the constructed new word $\Rightarrow$ therefore, it would never be in the list

	R	U	G	B	Y
					T
					E
3	O	F	F	E	R
4	C	L	E	A	R
5	P	H	L	C	X

Diagonalization Method & Infinities

- **Question** : is the set $\mathbb{R}$ of real numbers **countable** ? or does it have the same size as $\mathbb{N}$
 - Let's **assume that** we can enumerate all real numbers

1	.	1	0	0	0	...
2	.	1	1	0	0	..
3	.	2	1	2	3	...
4	.	3	1	4	3	...
5	.	..	..	..	...	..
..	.	..	..	..	..	...

Diagonalization Method & Infinities

- **Question** : is the set $\mathbb{R}$ of real numbers **countable** ? or does it have the same size as $\mathbb{N}$

- Let's **assume that** we can **enumerate all** real numbers
- Using the diagonalization method:
 - We can always generate new **real** numbers that are not inside the table

1	.	1	0	0	0	...
2	.	1	1	0	0	..
3	.	2	1	2	3	...
4	.	3	1	4	3	...
5	.	..	..	..	...	..
..	.	..	..	..	..	...

Diagonalization Method & Infinities

- **Question** : is the set $\mathbb{R}$ of real numbers **countable** ? or does it have the same size as $\mathbb{N}$

- Let's **assume that** we can **enumerate all** real numbers
- Using the diagonalization method:

.4357 ...

real

table

1	.	1	0	0	0	...
2	.	1	1	0	0	..
3	.	2	1	2	3	...
4	.	3	1	4	3	...
5	.	..	..	..	...	..
..	.	..	..	..	..	...

Diagonalization Method & Infinities

- **Question** : is the set $\mathbb{R}$ of real numbers **countable** ? or does it have the same size as $\mathbb{N}$

The set of real numbers $\mathbb{R}$ is uncountable

- We can always generate new **real** numbers that are not inside the table

					0	0	...
					0	0	..
2	.	1	1	2	3	...	
3	.	2	1	2	3	...	
4	.	3	1	4	3	...	
5	.	..	..	..	...	104	

Diagonalization Method & Infinities



- **Important Questions**

- Question 1 : is the language of Turing Machine :
 - Finite or Infinite ?
 - Countable or Uncountable ?
- Question 2 :
 - Can we solve all problems ? Are all problems computable ?
Can all problems represented by Turing Machines ?

Diagonalization Method & Infinities



- **Important Questions**

- Question 1 : is the language of Turing Machine :
 - It is infinite (You just keep adding states....)
 - But :
 - Each TM has :
 - Finite number of states
 - Finite Transitions
 - $\Rightarrow$ Each TM can be encoded as a finite string which itself can be converted to a natural number.

Diagonalization Method & Infinities



- **Important Questions**

- Question 1 : is the language of Turing Machine :
 - It is infinite
 - Another proof for TM as a countable and infinite set:
 - TMs can be represented/encoded as a string, therefore, it is a subset of the language Σ^*
 - We can enumerate strings of length 1 { 0, 1}, then strings of length 2, then strings of length 3 and so on...

Diagonalization Method & Infinities

- Important Questions

- Question 1 : is the language of Turing Machine .

The number of Turing Machines is infinite and countable

- Finite number of states
 - Finite Transitions
 - $\Rightarrow$ Each TM can be encoded as a finite string which itself can be converted to a natural number.

Diagonalization Method & Infinities

- Important Questions

- Question 2 :

- Can we solve all problems ? Are all problems computable ?
Can all problems represented by Turing Machines ?
 - All problems = All languages
 - The set of all languages can be represented as $P(\Sigma^*)$
 - $\emptyset$, $\{1\}$, $\{0\}$, $\{1,11,111,\dots\}$, $\{0,00,000,\dots\}$, $\{10,100,1000,\dots\}$

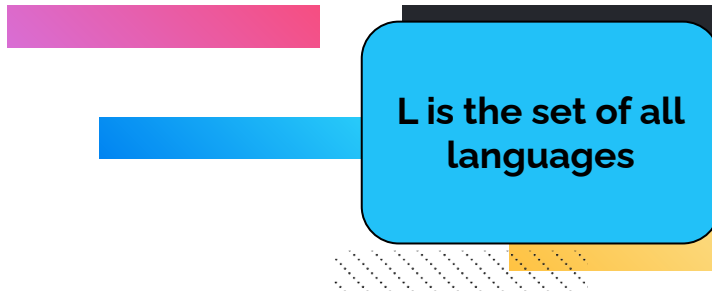
Diagonalization Method & Infinities

- **Important Questions** : Can we solve all problems ? Are all problems computable ?
 - Let B be the set of all **infinite binary sequences**.
 - 0000110000 ...
 - 0101010000 ...
 - 111111111111 ...
 - 101111110011 ...
 - ...

Diagonalization Method & Infinities

- **Important Questions** : Can we solve all problems ? Are all problems computable ?
 - Let **B** be the set of all **infinite binary sequences**. We can show that B is uncountable by using a **proof by diagonalization** using the same way used for real numbers.
 - Let **L** be the set of all languages over alphabet Σ
 - We need to prove that B (which is uncountable) has the same size as L in order to conclude that L is uncountable

Diagonalization Method & Infinities



L is the set of all languages

- **Important Questions** : Can we solve all problems ? Are all problems computable ?
 - **We need to find a correspondence function between B and L**
 - Let $\Sigma^* = \{s_1, s_2, s_3, \dots\}$ such that s_1, s_2, s_3 are just words from the language.
 - A is a language $\mathbf{A}, \in L$,
 - A a unique sequence in B.
 - The i^{th} bit of sequence is a 1 if $s_i \in A$ and is a 0 if s_i not in A,

Diagonalization Method & Infinities

- **Important Questions** : Can we solve all problems ? Are all problems computable ?

$$\begin{array}{l} \Sigma^* = \{ \epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots \} ; \\ A = \{ \quad 0, \quad \quad 00, 01, \quad \quad \quad 000, 001, \dots \} ; \\ \chi_A = \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \dots \end{array}$$

- ϵ not in A ? $\rightarrow 0$
- 1 in $A \rightarrow 1$
- 00 not in $A \rightarrow 0$

■ Until we create X_A which is the **characteristic sequence** of A

Diagonalization Method & Infinities

- Impo

com

Σ^*

A

χ_A

○

○

○

00 not in $A \rightarrow 0$

■ Until we create χ_A which is the **characteristic sequence** of A

Diagonalization Method & Infinities

- **Important Questions** : Can we solve all problems ? Are all problems computable ?

o

**The set of all languages is infinite and uncountable
Vs.
The number of Turing Machines is infinite and countable**

Diagonalization Method & Infinities

- **Important Questions** : Can we solve all problems ? Are all problems computable ?

- **There are languages that are not recognizable by turing machines**
i.e.
There are problems that they are not algorithmically computable

Diagonalization Method & Infinities



- **Last Question :**
 - We have seen that :
 - Acceptance in regular is solvable/computable
 - Acceptance for Context-Free Grammar is also solvable.
(Convert to Chomsky Normal form ...)
 - What about the Acceptance Problem for Turing Machine ?

Diagonalization Method & Infinities

- **Acceptance Problem for Turing Machine :**

- $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}.$
- We assume that A_{TM} is decidable and obtain a contradiction.

Suppose that H is a decider for A_{TM} such that.

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w \\ \text{reject} & \text{if } M \text{ does not accept } w. \end{cases}$$

Diagonalization Method & Infinities

- **Acceptance Problem for Turing Machine :**

- We construct a new Turing machine D with H as a subroutine. But, D does the opposite of H
- $D =$ "On input $\langle M \rangle$, where M is a TM:
 - *Run H on input $\langle M, \langle M \rangle \rangle$.*
 - *Output the opposite of what H outputs.*

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ does not accept } \langle M \rangle \\ \text{reject} & \text{if } M \text{ accepts } \langle M \rangle. \end{cases}$$

Diagonalization Method & Infinities

- **Acceptance Problem for Turing Machine :**
 - In English :
 - H accepts $\langle M, w \rangle$ exactly when M accepts w .
 - D rejects $\langle M \rangle$ exactly when M accepts **$\langle M \rangle$** .
 - D **rejects** $\langle D \rangle$ exactly when D **accepts** $\langle D \rangle$?

$$D(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ does not accept } \langle M \rangle \\ \text{reject} & \text{if } M \text{ accepts } \langle M \rangle. \end{cases}$$

Diagonalization Method & Infinities

•

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\dots$	$\langle D \rangle$	$\dots$
M_1	<u>accept</u>	reject	accept	reject		accept	
M_2	accept	<u>accept</u>	accept	accept		accept	
M_3	reject	reject	<u>reject</u>	reject	$\dots$	reject	$\dots$
M_4	accept	accept	reject	accept			
$\vdots$			$\vdots$				
D	reject	reject	accept	reject			
$\vdots$			$\vdots$				

$\dots$ pt $\langle M \rangle$

M2 accepts $\langle M_2 \rangle$,
Therefore
D rejects $\langle M_2 \rangle$

Diagonalization Method & Infinities

- | | $\langle M_1 \rangle$ | $\langle M_2 \rangle$ | $\langle M_3 \rangle$ | $\langle M_4 \rangle$ | $\dots$ | $\langle D \rangle$ | $\dots$ |
|----------|-----------------------|-----------------------|-----------------------|-----------------------|----------|---------------------|---------------------------------|
| M_1 | <u>accept</u> | reject | accept | reject | | accept | |
| M_2 | accept | <u>accept</u> | accept | accept | $\dots$ | accept | $\dots$ |
| M_3 | reject | reject | <u>reject</u> | reject | | reject | |
| M_4 | accept | accept | reject | <u>reject</u> | | accept | |
| $\vdots$ | | | $\vdots$ | | $\ddots$ | | |
| D | reject | reject | accept | accept | | <u>?</u> | |
| $\vdots$ | | | $\vdots$ | | | | $\ddots$ pt $\langle M \rangle$ |

Diagonalization Method & Infinities

- | | $\langle M_1 \rangle$ | $\langle M_2 \rangle$ | $\langle M_3 \rangle$ | $\langle M_4 \rangle$ | $\dots$ | $\langle D \rangle$ | $\dots$ |
|----------|-----------------------|-----------------------|-----------------------|-----------------------|---------|---------------------|---------|
| M_1 | <u>accept</u> | reject | accept | reject | | accept | |
| $\vdots$ | | $\vdots$ | | | | | $\dots$ |

pt $\langle M \rangle$

Which is a contradiction, therefore, The acceptance problem for Turing machine is not decidable

Proving by Reducibility

- **Problem** : Compute the area of a given square
 - The problem can be **reduced to** finding the width of the square
- A reduction is a way of converting one problem to another problem in such a way that a solution to the second problem can be used to solve the first problem
- **Reducibility** always involves two problems, which we call A and B. If A reduces to B, we can use a solution to B to solve A

Undecidability of the Halting Problem

- **Problem** : Can we decide whether turing machine halts on a given input either by accepting or rejecting without looping forever ?
- In the same way : Can we prove that a C/Pascal/Python program terminates on a given input ?
 - Question from the first lecture ?
- This is known as the **Halting Problem**

```
input n;  
assume n>1;  
while (n !=1) {  
    if (n is even)  
        n := n/2;  
    else  
        n := 3*n+1;  
}
```

Undecidability of the Halting Problem

- **Problem** : Halting Problem
- Language Representation :
 - $\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w \}$
- Proof :
 - We use the undecidability of A_{TM} to prove the undecidability of the halting problem by reducing A_{TM} to HALT_{TM}

Undecidability of the Halting Problem

- **Problem** : Halting Problem
- Let's assume :
 - S is the Turing Machine for the acceptance problem.
 - R is the Turing Machine for the Halting Problem

Undecidability of the Halting Problem

- **Problem** : Halting Problem
- Let's assume :
 - S is the Turing Machine for the acceptance problem.
 - R is the Turing Machine for the Halting Problem
- $S =$ "On input $\langle M, w \rangle$, an encoding of a Turing machine M and a string w :
 - Run TM R on input $\langle M, w \rangle$
 - If R rejects, reject .
 - If R accepts, simulate M on w until it halts.
 - If M has accepted, accept ; if M has rejected, reject .

Undecidability of the Halting Problem

- **Problem** : Halting Problem
- Let's assume :
 - S is the Turing Machine for the acceptance problem.
 - R is the Turing Machine for the Halting Problem
- $S =$ "On input $\langle M, w \rangle$, an encoding of a Turing machine M and a string w :
 - Run TM R on input $\langle M, w \rangle$
 - If R rejects, reject .
 - If R accepts, simulate M on w until it halts.
 - If M has accepted, accept ; if M has rejected, reject .

This shows clearly that the Acceptance Problem is Decidable, which is a contradiction,

Therefore,

The halting problem is undecidable

More Undecidable Questions

- E_{TM} is undecidable
- EQ_{TM} is undecidable.
- ...

Post Correspondence Problem

- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of two string:

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$$

- The task is to make a list of these dominos (repetitions permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom

Post Correspondence Problem

- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of two string:

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$$

We have to start with this domino because the first letter of the numerator is the same as first letter of the denominator

Use dominos (repetitions permitted) so that the string of the symbols on the top is the same as the string of the symbols on the bottom

Post Correspondence Problem

- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of two string:

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$$

- The task is to make a list of these dominos (repetitions permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom

$$\left[\frac{a}{ab} \right]$$

Post Correspondence Problem

- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of two string:

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$$

- The task is to make a list of these dominos (repetitions permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on

$$\left[\frac{a}{ab} \right]$$

We need a domino with a leading b on the numerator

Post Correspondence Problem

- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of two string:

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$$

- The task is to make a list of these dominos (repetitions permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom

$$\left[\frac{a}{ab} \right] \left[\frac{b}{ca} \right] \left[\frac{ca}{a} \right] \left[\frac{a}{ab} \right] \left[\frac{abc}{c} \right]$$

Post Correspondence Problem

- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of two strings

**This instance of the problem is SOLVABLE.
COMPUTABLE, DECIDABLE**

- The task is to find a sequence of dominos (if permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom

$$\left[\begin{array}{c} a \\ ab \end{array} \right] \left[\begin{array}{c} b \\ ca \end{array} \right] \left[\begin{array}{c} ca \\ a \end{array} \right] \left[\begin{array}{c} a \\ ab \end{array} \right] \left[\begin{array}{c} abc \\ c \end{array} \right]$$

Post Correspondence Problem

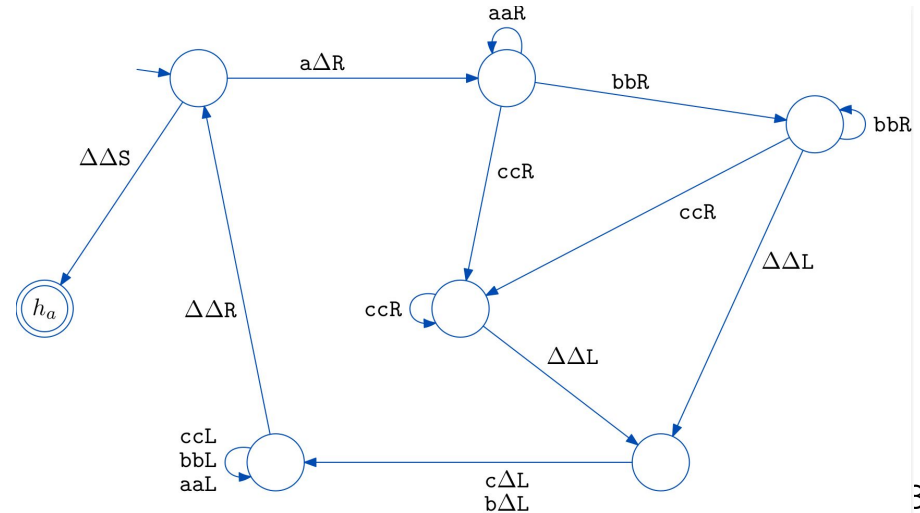
- Post Correspondence Problem (PCP) : Given a collection of dominos in the form of tiles $\left[\begin{smallmatrix} \text{top} \\ \text{bottom} \end{smallmatrix} \right]$ $\left\{ \left[\begin{smallmatrix} a \\ ab \end{smallmatrix} \right], \left[\begin{smallmatrix} b \\ ca \end{smallmatrix} \right], \left[\begin{smallmatrix} ca \\ a \end{smallmatrix} \right], \left[\begin{smallmatrix} a \\ ab \end{smallmatrix} \right], \left[\begin{smallmatrix} abc \\ c \end{smallmatrix} \right] \right\}$.
- The task is to find a sequence of dominos (if permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom

$$\left[\begin{smallmatrix} a \\ ab \end{smallmatrix} \right] \left[\begin{smallmatrix} b \\ ca \end{smallmatrix} \right] \left[\begin{smallmatrix} ca \\ a \end{smallmatrix} \right] \left[\begin{smallmatrix} a \\ ab \end{smallmatrix} \right] \left[\begin{smallmatrix} abc \\ c \end{smallmatrix} \right]$$

TD9 - Solutions

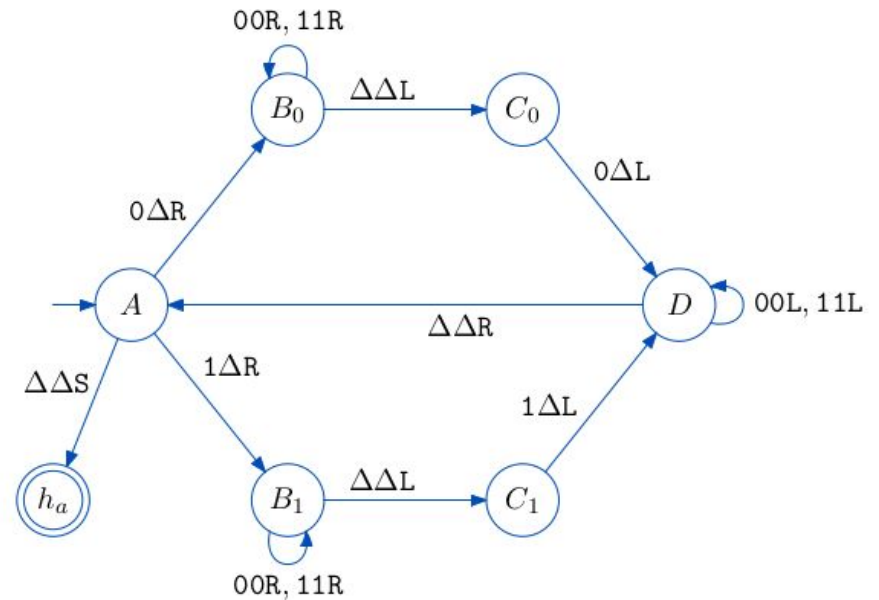
Let the the language $L = \{ a^i b^j c^k \}$ such that $i = j + k$.

- Write in **plain english english**, the algorithm fo
- Draw the Turing machine which recognizes L . (**NO USE OF PC**)
(*Replace S with R*)



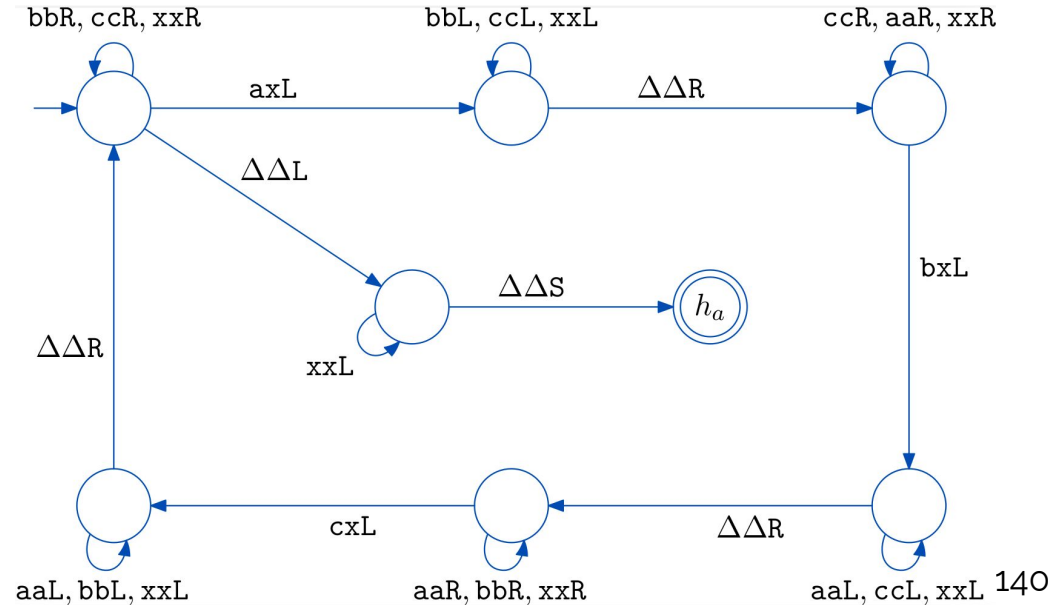
TD9 - Solutions

Draw the Turing Machine diagram for Even-Length Palindromes. (**NO USE OF PC**)



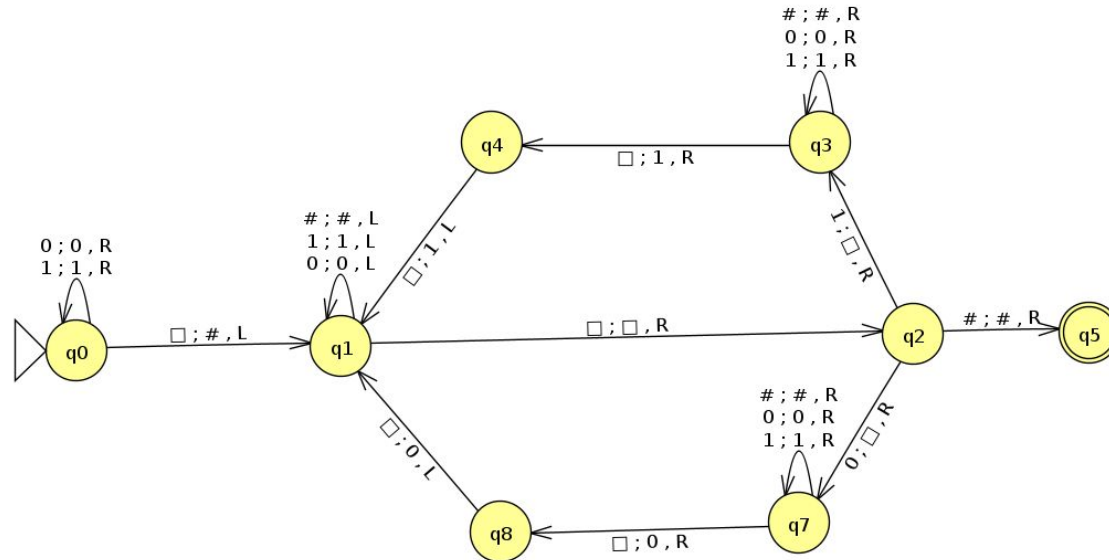
TD9 - Solutions

Draw a TM with alphabet $\{a, b, c\}$ that accepts all strings where the number of each character is the same
[the number of a = number of b = number of c] **[You can use your PC]**



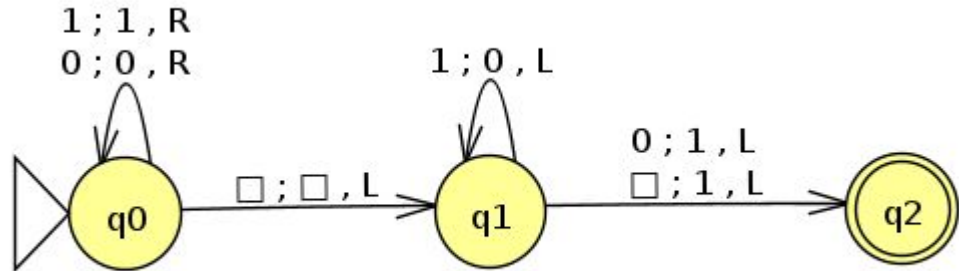
TD10 - Solutions

Construct a Turing machine that doubles each character in its input string. For example, if the input is 0100, then the machine should change its tape so it contains 00110000. Show all the states and transitions.



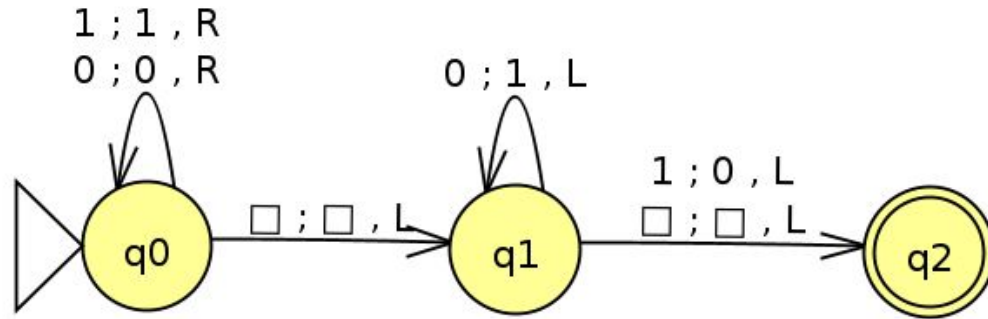
TD10 - Solutions

Turing Machine to Increment a number



TD10 - Solutions

Turing Machine to decrement a number :
We don't consider the cases of reaching negative numbers



TD10 - Solutions

Write the algorithm and design the draw machine using JFLAP for the following problem.

- Count the number of symbols in an input string and write the result as a binary number at the tape. After execution, the tape should contain only the result.

